

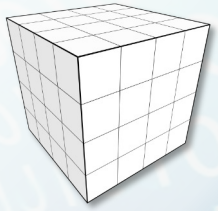


Section 4: ZFP Lossy and Lossless Compressor

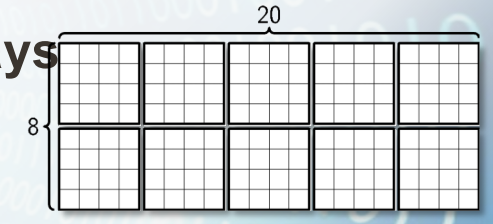
Peter Lindstrom, Lawrence Livermore National Laboratory

What is ZFP?

- **Answer #1**: An efficient floating **number format** for tiny tensors
  - Alternative to IEEE/SSE/AVX/tensor core registers, bfloats, posits, flexpoint, …



- **Answer #2**: An implementation of **multidimensional arrays**
  - Alternative to std::vector, Eigen/GSL/Kokkos/NumPy arrays, …



- **Answer #3**: A fast, **streaming compressor** for numerical data
  - Alternative to gzip, bzip2, blosc, fpzip, JPEG, …

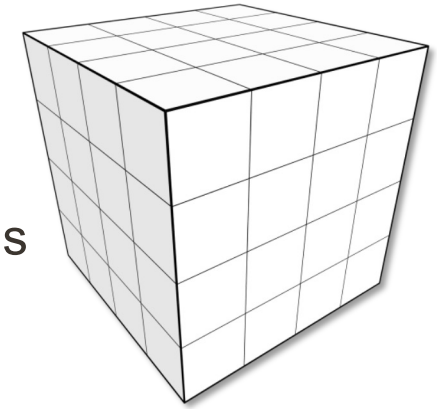


ZFP salient features

- Supports **integer** and **floating-point** multi-dimensional Cartesian data
- Supports **lossy** and **lossless** compression
- Supports **fixed rate**, absolute & relative **error tolerances**
- Supports **random access** reads & writes to array elements at block granularity
  - *d*-dimensional block is 4^d values
- Supports **massive data parallelism** though partitioning into independent blocks
- Is **extremely fast** with numerous CPU and GPU back-ends
- Was primarily designed for in-memory compressed storage
  - **Numerical computations** like PDEs, data analysis, visualization
  - Also **reduces data movement** in I/O, communication, CPU-GPU transfers

ZFP is a compressed number format for multi-dimensional floating-point arrays

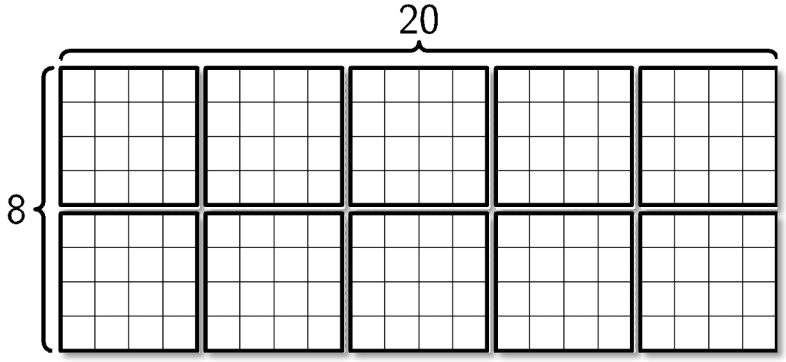
- ZFP compactly represents small blocks of real values
  - Encodes *d*-dimensional **block of 4^d values** as variable-length bit string
  - **Fixed-length code** obtained via bit string truncation
    - Analogous to FP rounding: 1/5 = 0.001100110011… ≈ 0.0011
    - "Common" blocks have shorter codes ⟹ less or no round-off error
  - **H/W friendly**: parallel integer additions and bitwise operations



- ZFP **replaces IEEE 754** as format for numerical computations
  - **Less storage** for same accuracy
  - **Higher accuracy** for same storage
  - **Fine-grained precision** selection

ZFP multi-dimensional arrays offer in-memory compressed storage with high-speed read and write access

- ZFP provides C++ classes for multi-dimensional arrays
  - Read & write **random access** at block granularity
    - Block decomposition is transparent to user
  - User specifies **memory footprint** or **error tolerance**
  - **Conventional API**: C++ operator overloading hides complexity of (de)compression
    - double a[n] ⇔ std::vector<double> a(n) ⇔ zfp::array<double> a(n, **bits_per_value**)
    - C, experimental NumPy APIs available



zfp's C++ compressed arrays can replace STL vectors and C arrays with minimal code changes

// example using STL vectors

```
std::vector<double> u(nx * ny, 0.0);
u[x0 + nx*y0] = 1;
for (double t = 0; t < tfinal; t += dt) {
        std::vector<double> du(nx * ny, 0.0);
        for (int y = 1; y < ny - 1; y++)
                for (int x = 1; x < nx - 1; x++) {
                        double uxx = (u[(x-1)+nx*y] - 2*u[x+nx*y] +
u[(x+1)+nx*y]) / dxx;
                        double uyy = (u[x+nx*(y-1)] - 2*u[x+nx*y] +
u[x+nx*(y+1)]) / dyy;
                        du[x + nx*y] = k * dt * (uxx + uyy);
                }
        for (int i = 0; i < u.size(); i++)
                u[i] += du[i];
}
```

// example using zfp arrays

```
zfp::array2<double> u(nx, ny, bits_per_value);
u(x0, y0) = 1;
for (double t = 0; t < tfinal; t += dt) {
        zfp::array2<double> du(nx, ny, bits_per_value);
        for (int y = 1; y < ny - 1; y++)
                for (int x = 1; x < nx - 1; x++) {
                        double uxx = (u(x-1, y) -
2*u(x, y) + u(x+1, y)) / dxx;
                        double uyy = (u(x, y-1) -
2*u(x, y) + u(x, y+1)) / dyy;
                        du(x, y) = k * dt * (uxx +
uyy);
                }
        for (int i = 0; i < u.size(); i++)
                u[i] += du[i];
}
```

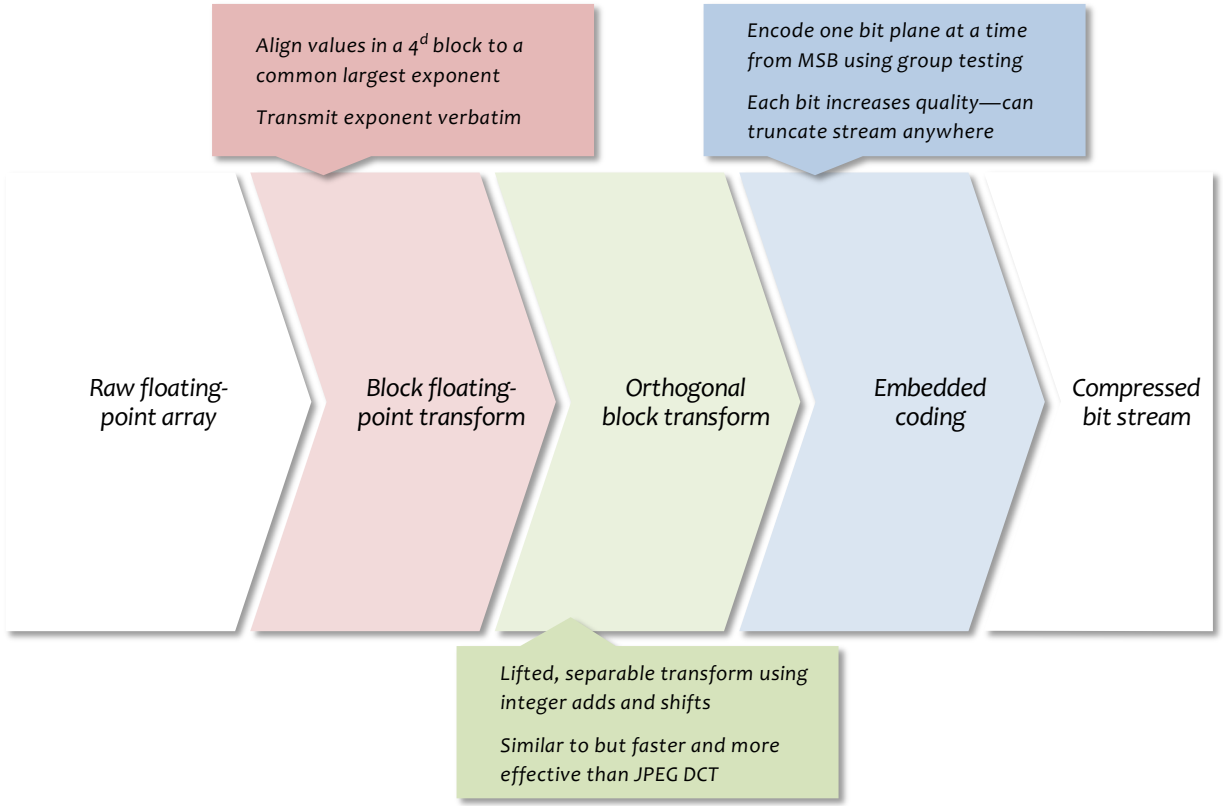
- required changes
- optional changes for improved readability

ZFP supports fast, parallel (de)compression of whole arrays

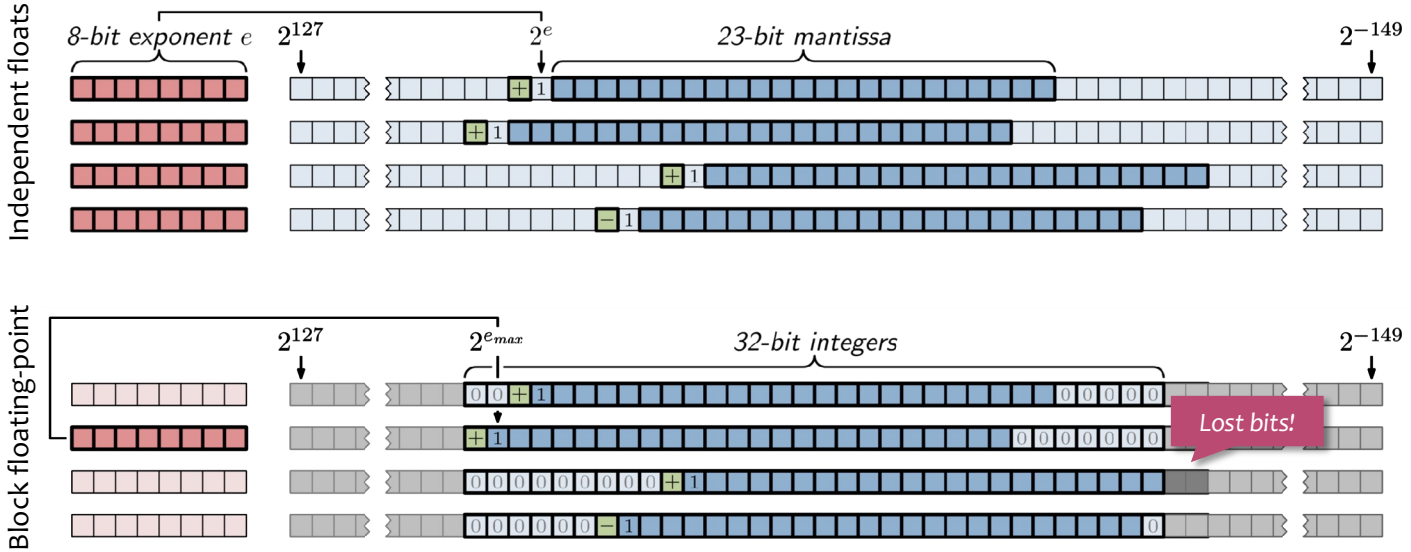
- ZFP supports streaming compression for I/O, communication, storage, …
  - Supports absolute and relative **error tolerances** and **lossless** compression
  - Back-ends: serial, **OpenMP**, **CUDA**, **HIP**, **FPGA** (AVX, SYCL, NEC VE through 3rd party)
    - Up to **700 GB/s** parallel throughput
  - Language bindings: **C**, **C++**, **Python**, **Fortran** (Julia, Rust, WebAssembly through 3rd party)



The ZFP encoder is comprised of three distinct components



ZFP uses a block floating-point representation that allows treating values as integers



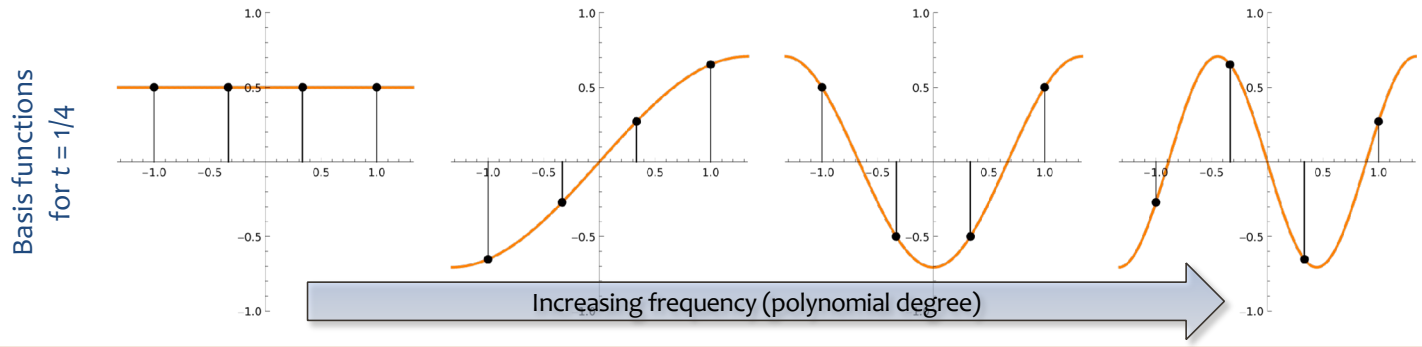
- Block floating-point: Align values to a common, largest exponent e_{max}
  - Exponent is transmitted uncompressed
  - Floats (doubles) are converted to 32-bit (64-bit) signed integers

Decorrelation: ZFP decorrelates values in a block using a separable orthogonal transform

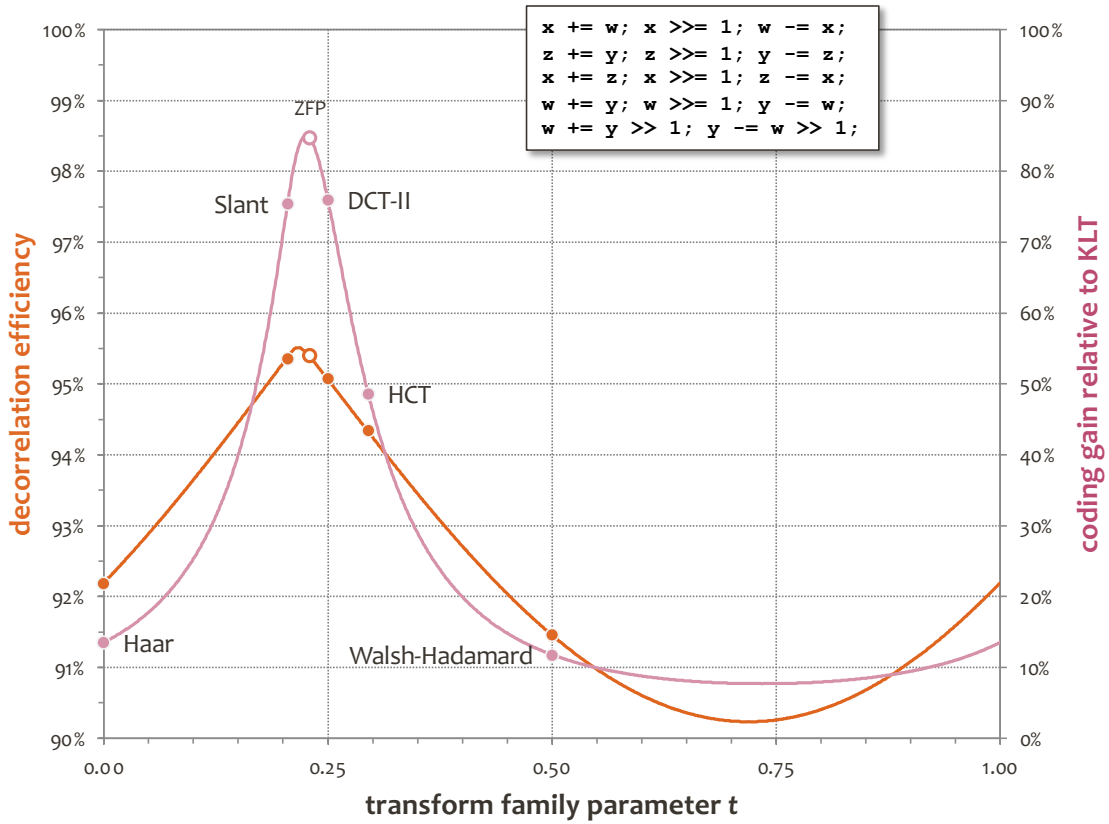
$$\underbrace{\begin{pmatrix} \hat{f}_1 \\ \hat{f}_2 \\ \hat{f}_3 \\ \hat{f}_4 \end{pmatrix}}_{\text{coefficients}} = \underbrace{\frac{1}{2}\begin{pmatrix} 1 & 1 & 1 & 1 \\ c & s & -s & -c \\ 1 & -1 & -1 & 1 \\ s & -c & c & -s \end{pmatrix}}_{\text{orthogonal transform}} \underbrace{\begin{pmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \end{pmatrix}}_{\text{block}}$$

Free parameter t

$$s = \sqrt{2}\sin\tfrac{\pi}{2}t \qquad c = \sqrt{2}\cos\tfrac{\pi}{2}t$$

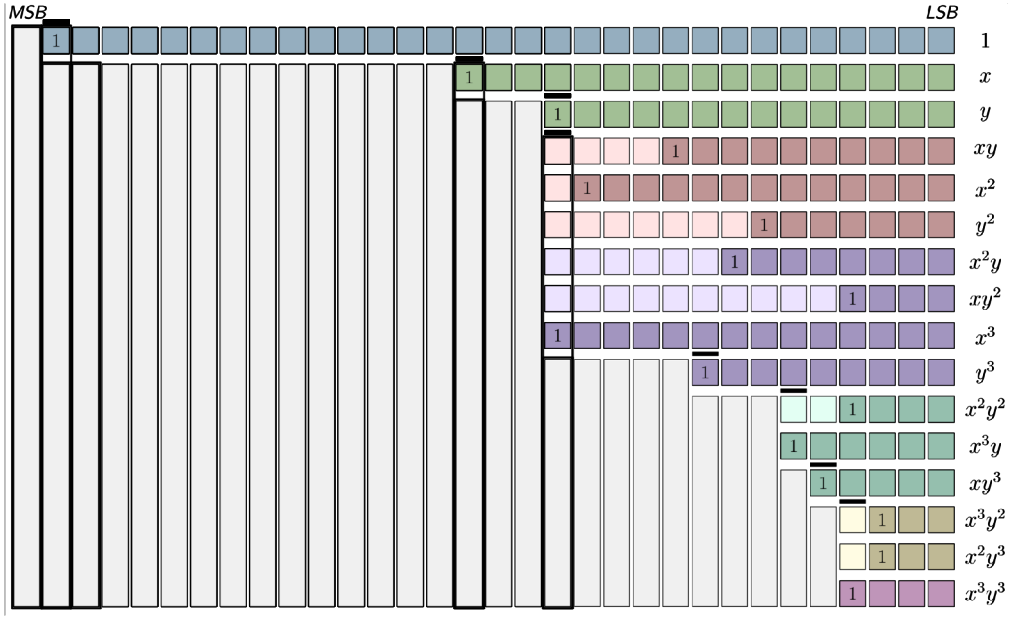


ZFP's integer transform is efficient, effective, and well-suited for h/w implementation

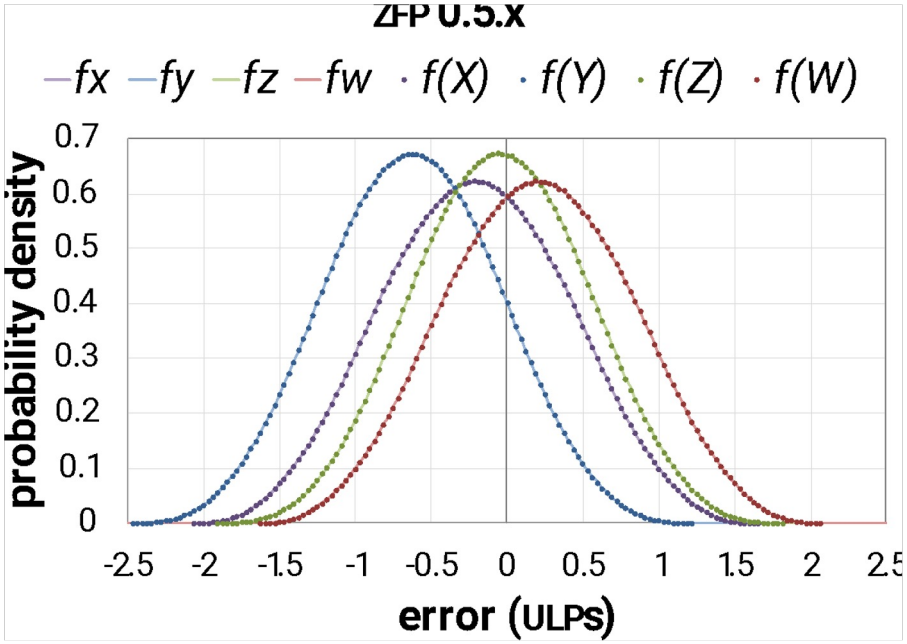


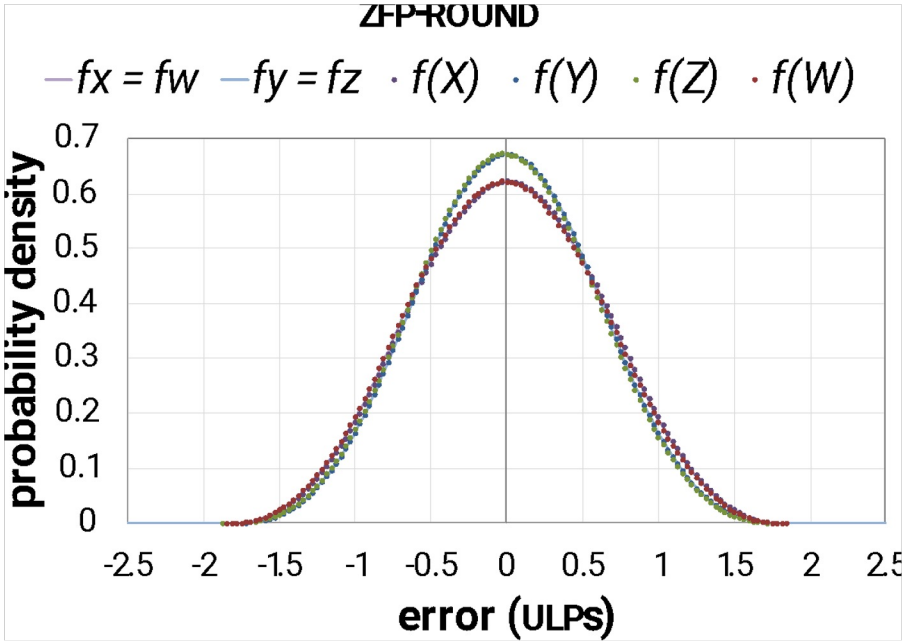
Coding & approximation: Compression is achieved by grouping leading zeros and truncating least significant bits

- Group test (aka. pool test): Are all bits within a group from the same bit plane zero?
  - If (test passes), output single zero bit and proceed to next bit plane
  - If **no** (test fails), output bits until one-bit emitted, then group test remaining bits
  - Encode coefficients one bit plane at a time, from MSB to LSB

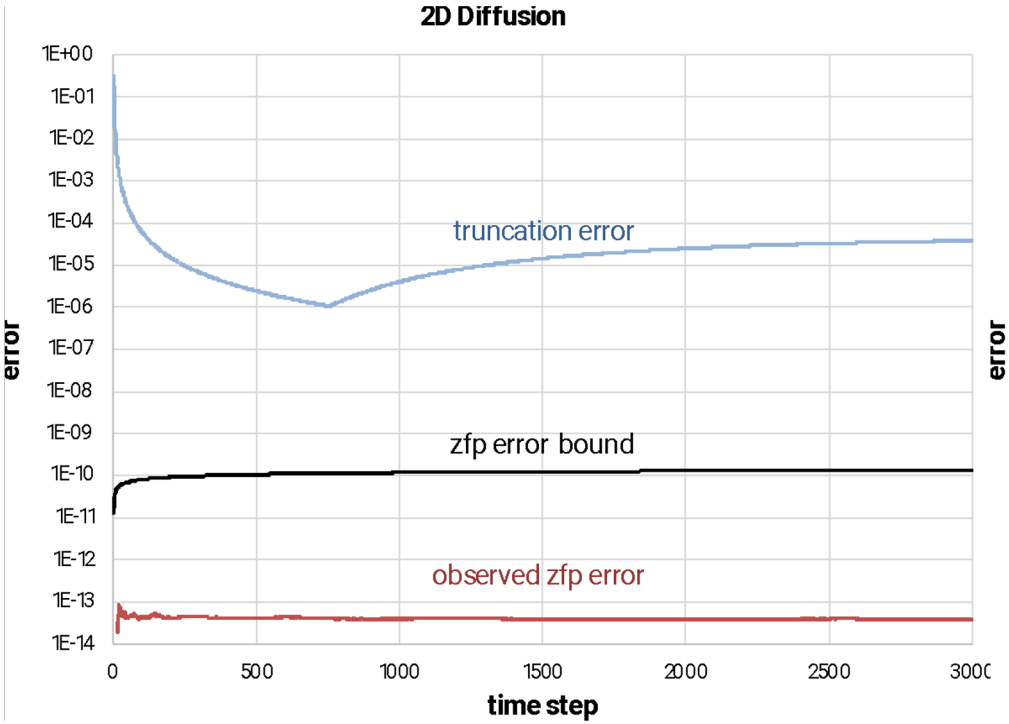


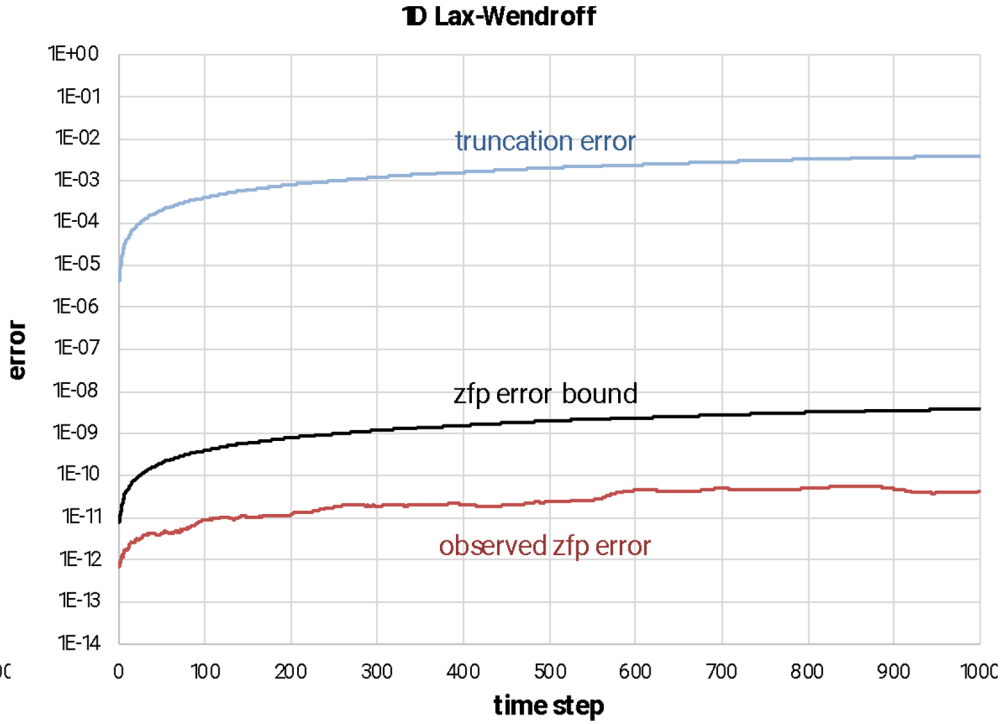
ZFP errors are Gaussian, bounded in (−5/2, +5/2) ULPs, and unbiased in ZFP 1.0.0



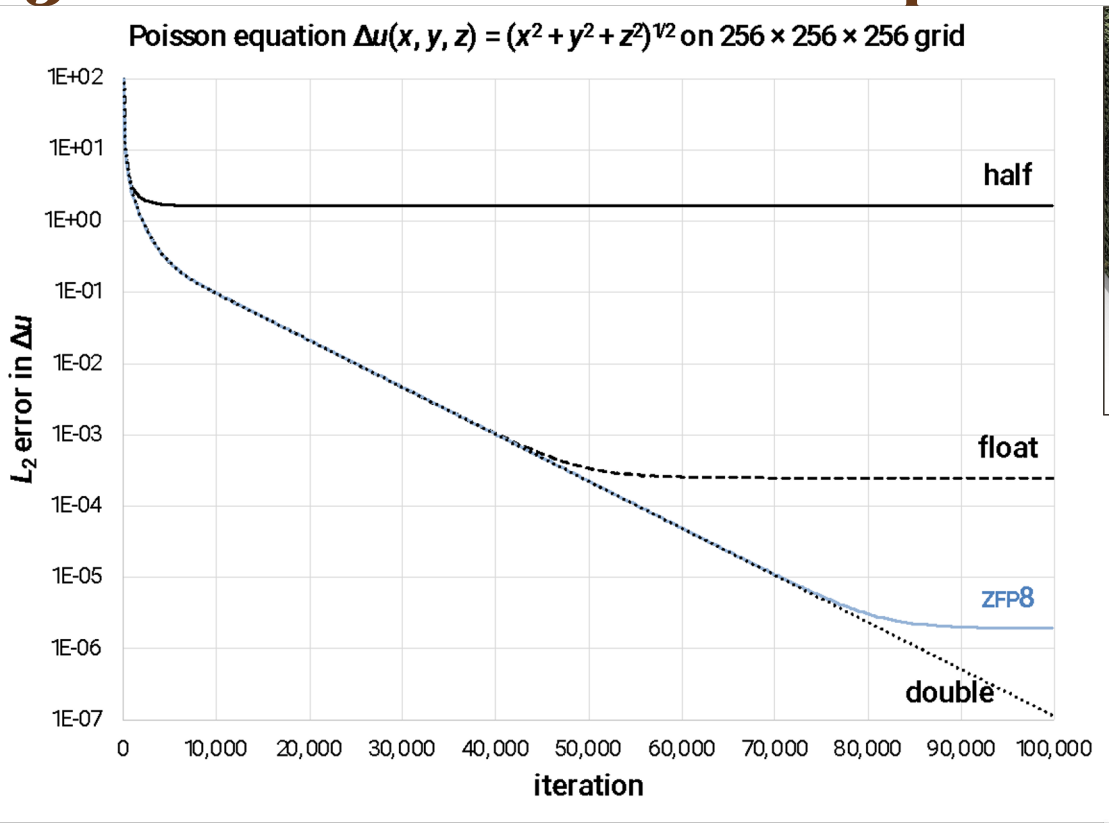


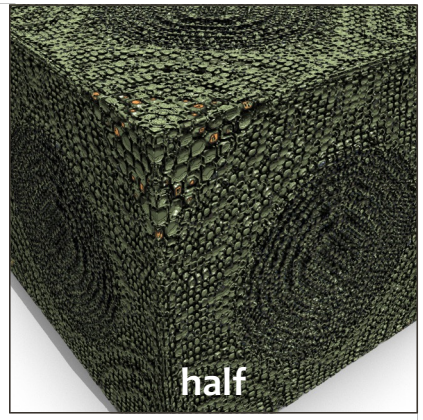
We have developed rigorous error bounds for ZFP, both for static data and in iterative methods



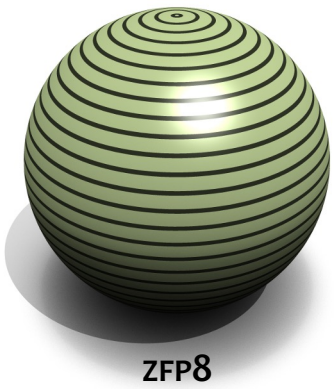


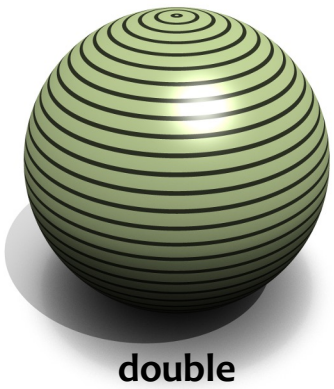
8-bit ZFP yields 2 orders of magnitude higher accuracy than 32-bit floats in Poisson equation solver



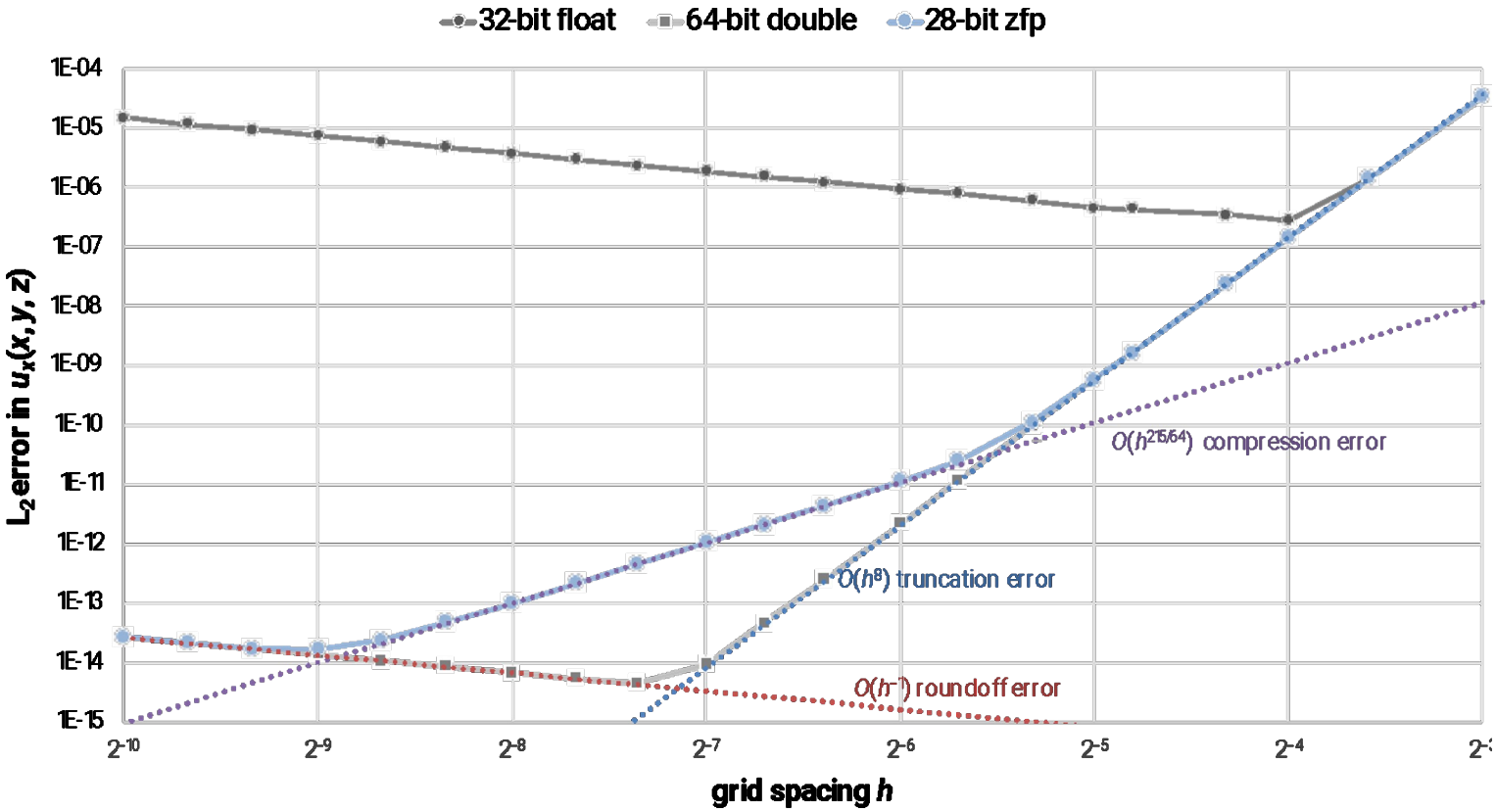




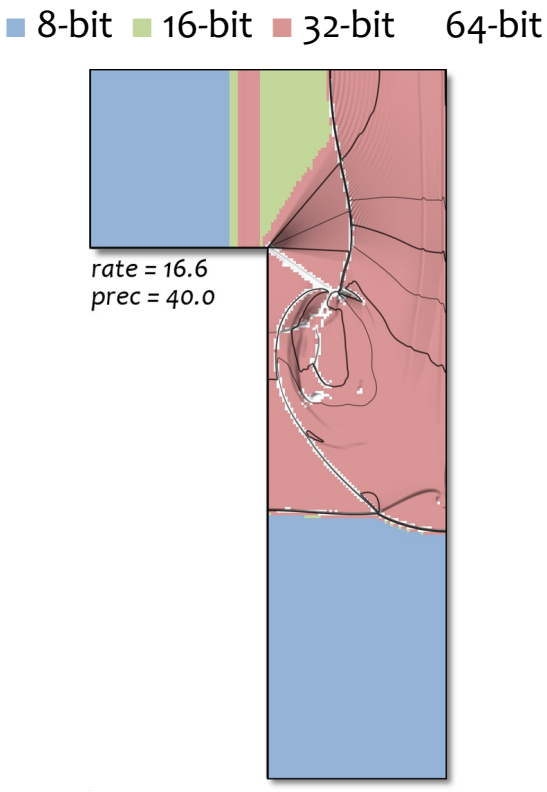


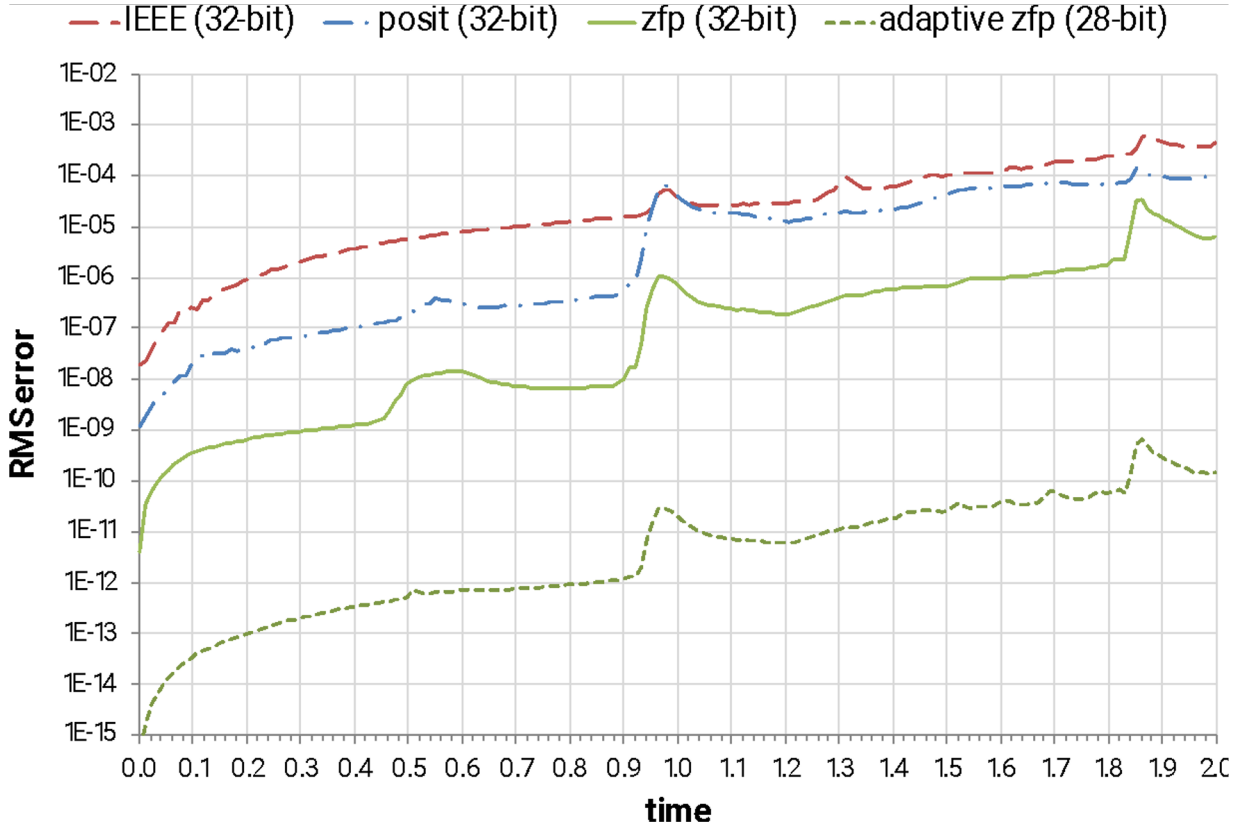


Contrary to conventional floating point, finite-difference accuracy using ZFP *increases* with grid resolution

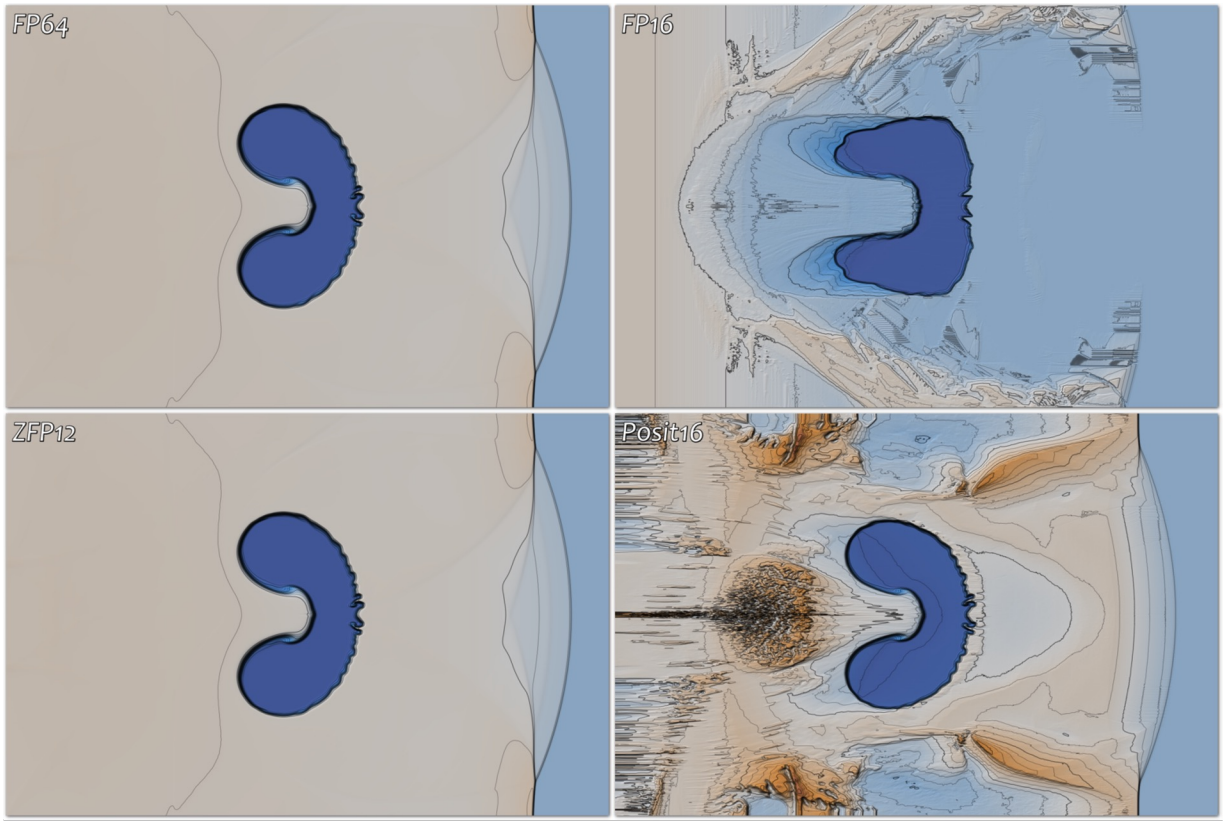


Adaptive-rate ZFP increases accuracy over IEEE float by 6 orders of magnitude while using less storage

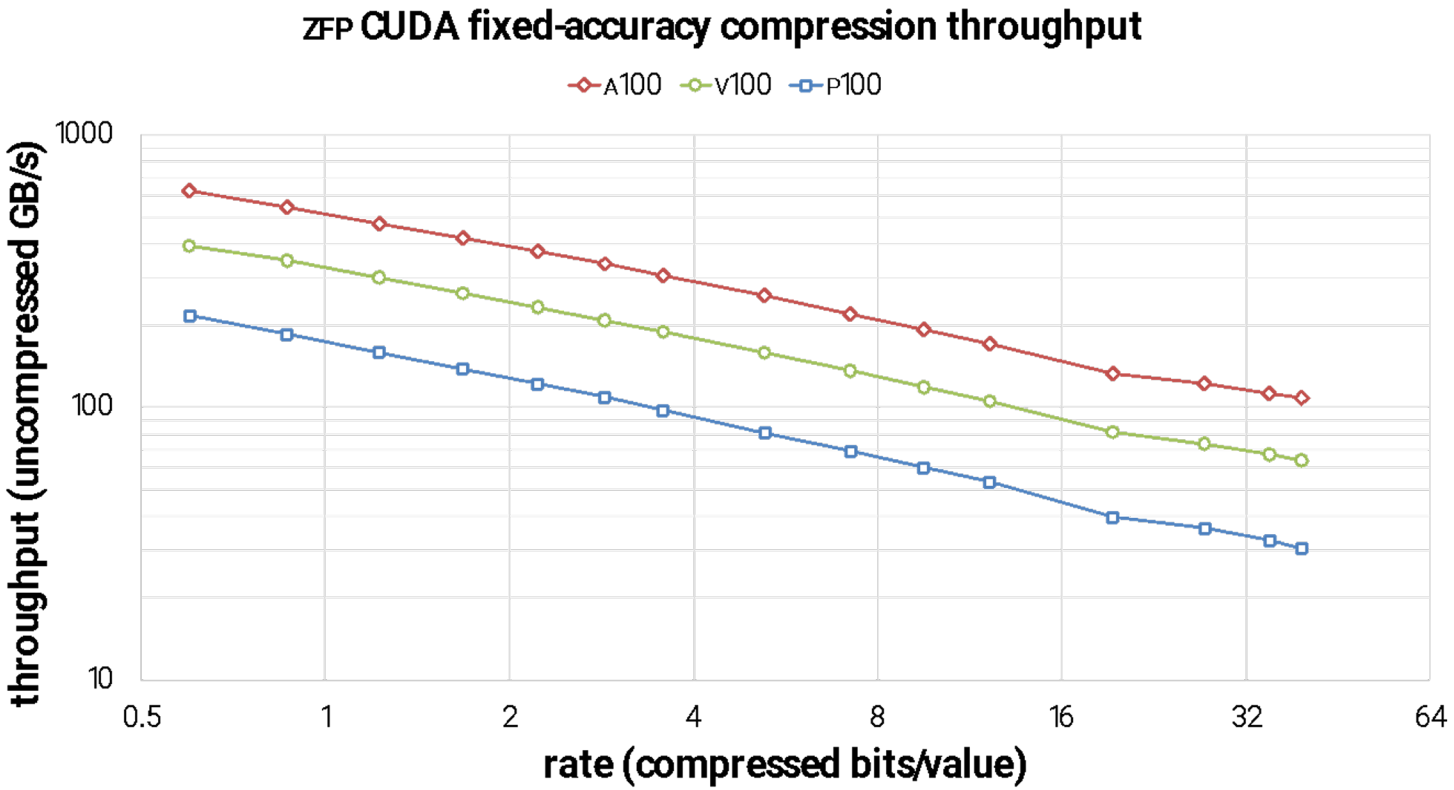




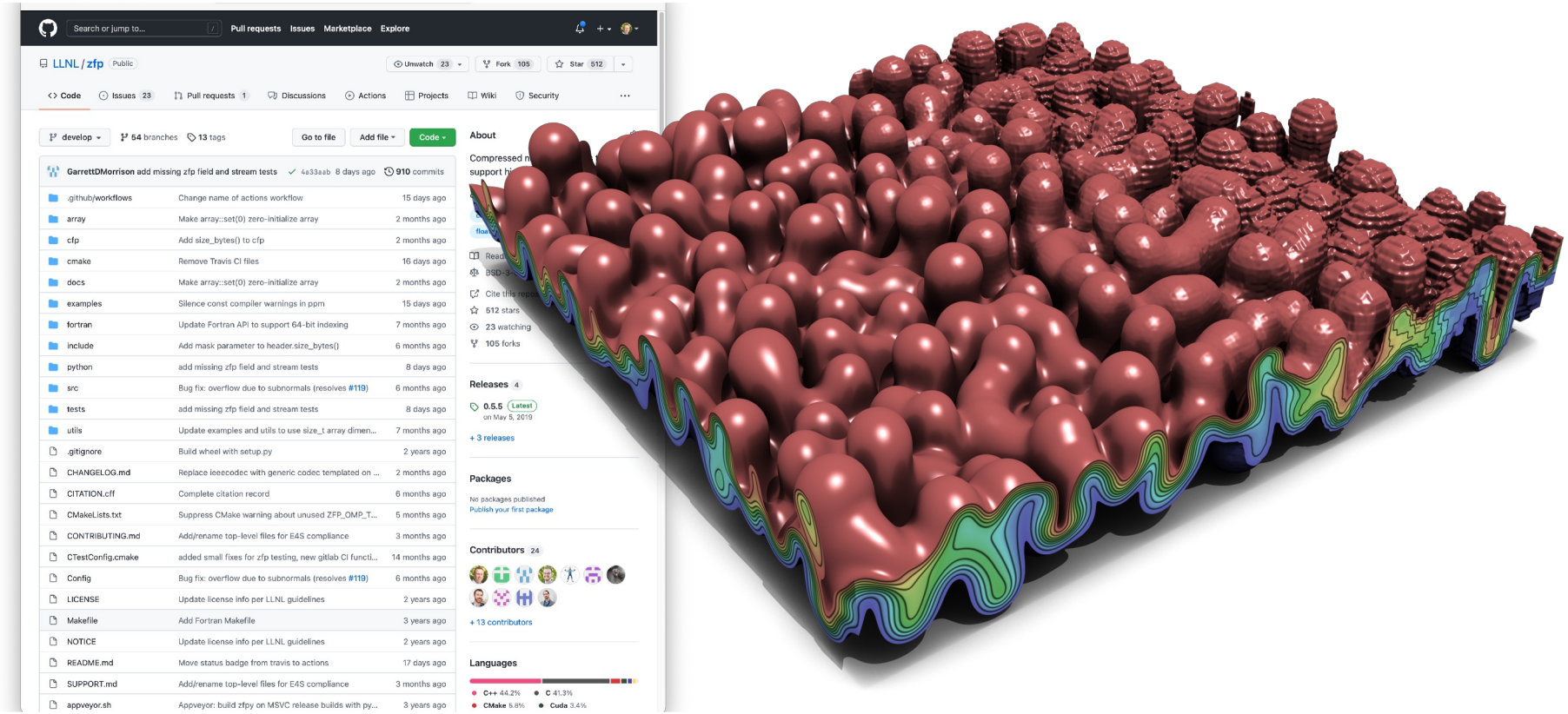
PDE computations using 12-bit ZFP are qualitatively identical to 64-bit IEEE double precision



zfp GPU compression achieves up to 700 GB/s throughput



ZFP is available as open source on GitHub: https://github.com/LLNL/zfp





Acknowledgment

This research was supported by the Exascale Computing Project (17-SC-20-SC), a joint project of the U.S. Department of Energy's Office of Science and National Nuclear Security Administration, responsible for delivering a capable exascale ecosystem, including software, applications, and hardware technology, to support the nation's exascale computing imperative.

Disclaimer

This document was prepared as an account of work sponsored by an agency of the United States government. Neither the United States government nor Lawrence Livermore National Security, LLC, nor any of their employees makes any warranty, expressed or implied, or assumes any legal liability or responsibility for the accuracy, completeness, or usefulness of any information, apparatus, product, or process disclosed, or represents that its use would not infringe privately owned rights. Reference herein to any specific commercial product, process, or service by trade name, trademark, manufacturer, or otherwise does not necessarily constitute or imply its endorsement, recommendation, or favoring by the United States government or Lawrence Livermore National Security, LLC. The views and opinions of authors expressed herein do not necessarily state or reflect those of the United States government or Lawrence Livermore National Security, LLC, and shall not be used for advertising or product endorsement purposes.